



REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
ASIGNATURA: INGENIERÍA DE SOFTWARE II

**GESTIÓN CUANTITATIVA DE LA INCERTIDUMBRE EN LA ESTIMACIÓN DE
SOFTWARE: ANÁLISIS EXHAUSTIVO DE MÉTRICAS EMPÍRICAS DE
LIDERAZGO TÉCNICO**

Evaluación — Ejercicio 4 (Bonus Unidad III)

Profesor:

Mg. Félix Márquez

Integrantes:

Gómez Andrés V-31.085.717

Ramírez Alexmary V-31.809.930

Silva José V-30.810.283

Vallenilla Daniel V-31.159.105

Ciudad Guayana, septiembre de 2026

Introducción

La estimación tradicional en ingeniería de software suele basarse en horas-hombre y conjeturas cualitativas, lo que precipita con frecuencia la denominada *falacia de la planificación* (Kahneman & Tversky, 1979). Proyectar plazos rígidos sin considerar la variabilidad intrínseca del desarrollo degrada la previsibilidad del proyecto y erosiona la confianza de los interesados. Frente a ello, el liderazgo técnico debe asumir un principio clave: **estimar no es adivinar el futuro, sino cuantificar y gestionar la incertidumbre a través de evidencia empírica.**

El presente informe desarrolla el análisis crítico del Ejercicio 4 (Bonus Unidad III), estableciendo cuatro métricas indispensables para el inicio de nuevos proyectos: el flujo de valor (velocidad y tiempo de ciclo) con Simulación de Monte Carlo, la deuda técnica como pasivo financiero generador de intereses, la minería del historial de Git y la evaluación de la estabilidad y madurez del equipo.

Desarrollo: Marco Métrico para el Liderazgo Técnico

1. Métricas de Flujo: Velocidad, Tiempo de Ciclo y Modelado Probabilístico con Simulación de Monte Carlo

En el marco de la ingeniería de software actual, persistir en estimar esfuerzos a través de horas-hombre individuales constituye una falacia analítica. Las horas-hombre confunden esfuerzo teórico con duración de calendario, asumiendo erróneamente que una tarea de cuarenta horas puede distribuirse linealmente sin fricciones. Dicha aproximación desestima variables críticas tales como la sobrecarga de cambio de contexto (*context switching*), las esperas asincrónicas en revisiones de pares, la latencia en tuberías de integración continua y los bloqueos originados por dependencias externas. Frente a estas distorsiones, el liderazgo técnico debe recurrir a métricas de flujo fundamentadas en la teoría de colas y el pensamiento Lean (Vacanti, 2015):

Tiempo de Ciclo (Cycle Time): Representa el lapso exacto transcurrido desde el momento en que una unidad de trabajo (historia de usuario, defecto o requerimiento) abandona el inventario priorizado y entra formalmente en estado de desarrollo activo (fase *In Progress*), hasta que satisface plenamente la Definición de Terminado (*Definition of Done*) y queda desplegada o verificada en un entorno de integración. A diferencia del *Lead Time*, que cuantifica la ventana total desde la concepción de la idea e incluye tiempos pasivos en el backlog, el *Cycle Time* mide exclusivamente la eficiencia y fluidez del proceso técnico.

Rendimiento Operativo (Throughput o Velocidad Empírica): Cuantifica la cantidad de unidades de valor completadas satisfactoriamente en una unidad de tiempo fija (por ejemplo, el número de ítems de backlog o Story Points finalizados en un sprint de dos semanas). Para el líder de equipo, el throughput adquiere validez predictiva únicamente cuando se preserva su historial cronológico continuo y se calcula su dispersión estadística (media, varianza y desviación estándar), descartando el uso de promedios simples aislados.

La articulación matemática de estas métricas habilita la aplicación de la **Simulación de Monte Carlo**. Las aproximaciones convencionales intentan proyectar fechas sumando los valores esperados de cada ítem de trabajo, incurriendo en el error estadístico de obviar que las duraciones de tareas no siguen una distribución normal o simétrica, sino distribuciones con

asimetría positiva hacia la derecha (tipo Weibull o Lognormal). Mediante Monte Carlo, el líder introduce el historial empírico de velocidad y tiempo de ciclo del equipo en un modelo computacional que simula el desarrollo del backlog miles de veces (típicamente entre 5.000 y 10.000 ejecuciones sintéticas aleatorias).

A partir de este procedimiento se genera una Función de Distribución Acumulada (CDF) que transforma radicalmente el diálogo con los comités directivos y clientes. El líder técnico prescinde de comprometer fechas deterministas frágiles ("el proyecto se entregará exactamente el 15 de noviembre") y formula acuerdos basados en percentiles de certidumbre estadística: "*Se registra un 50% de probabilidad de entregar las 45 historias en 4 semanas, pero un 85% de confianza estadística de completarlas en 5.5 semanas y un 95% de certeza en 6 semanas*". De este modo, la toma de decisiones incorpora márgenes de riesgo matemáticamente acotados y auditables.

Tabla 1

Matriz Comparativa de Enfoques de Planificación Temporal

Criterio de Evaluación	Enfoque Tradicional Determinista	Enfoque Empírico Probabilístico
Unidad analítica base	Horas-hombre / Estimación subjetiva	Throughput histórico y Cycle Time real
Naturaleza del compromiso	Fecha fija puntual e inflexible	Rango temporal acoplado a percentil (e.g., 85%)
Tratamiento de la varianza	Colchones arbitrarios de contingencia (+20%)	Distribución estocástica de Monte Carlo

2. La Deuda Técnica como Pasivo Financiero y su Amortización

La conceptualización formulada originalmente por Cunningham (1992) asimila la adopción intencionada o inadvertida de atajos de diseño con la contratación de un empréstito financiero. Desde la perspectiva de la economía de la ingeniería, la deuda técnica constituye un

pasivo financiero que genera un interés compuesto continuo. En el ámbito informático, el pago de dicho interés no se liquida mediante flujos monetarios, sino a través de una severa fricción operativa acumulativa:

- **Sobrecarga de complejidad cognitiva:** El tiempo que un ingeniero de software requiere para interpretar el flujo de datos, rastrear dependencias cruzadas y comprender el alcance de una modificación se expande de forma exponencial en códigos deteriorados.

- **Degradación paulatina de la velocidad de desarrollo:** A medida que la deuda técnica se incrementa, la tasa de entrega de nuevas características sufre una deceleración sistemática; una tarea que en una arquitectura limpia demandaría dos días de desarrollo requiere semanas cuando debe sortear capas de código espagueti.

- **Tasa de fallas y regresiones inesperadas:** La carencia de modularidad y acoplamiento severo genera el fenómeno de propagación de defectos, en el cual una alteración localizada desestabiliza módulos periféricos no vinculados funcionalmente.

Para cuantificar de manera objetiva este pasivo antes de iniciar o extender un proyecto, el líder técnico no debe depender de apreciaciones cualitativas, sino de herramientas analíticas de inspección estática (como SonarQube). Estas plataformas procesan el código base calculando métricas fundamentales de arquitectura: la *complejidad ciclomática de McCabe* (que mide el número de trayectorias linealmente independientes a través del grafo de control del software), el índice de duplicación textual y sintáctica, el acoplamiento eferente y aferente entre subsistemas y el porcentaje de cobertura de pruebas unitarias y de integración.

La estrategia de liderazgo ante este escenario exige institucionalizar una partida innegociable en la planificación de capacidades: **reservar formalmente entre un 15% y un 20% del esfuerzo operativo de cada ciclo o sprint exclusivamente para labores de amortización de deuda técnica.** Este presupuesto continuo se destina a la refactorización dirigida, la eliminación de código obsoleto, la actualización de bibliotecas de terceros y la automatización de pruebas de regresión. Estimar nuevos proyectos asumiendo una capacidad productiva del 100% sobre un sistema infectado de deuda técnica equivale a edificar sobre cimientos inestables: la deuda devorará la velocidad del equipo y tornará cualquier proyección temporal en una ficción insostenible.

3. Minería de Repositorios: Análisis del Historial de Git como Evidencia Forense

El repositorio de control de versiones de un proyecto de software almacena la cronología empírica más exhaustiva e incontestable de la interacción entre el equipo de desarrollo y los componentes arquitectónicos. Lejos de constituir un mero repositorio de almacenamiento, el historial de Git es un activo de datos forenses cuya explotación analítica suministra diagnósticos directos sobre la estabilidad del diseño (Tornhill, 2015). A través de instrucciones estructuradas de inspección profunda como:

```
git log --shortstat --pretty=format:'%h | %an | %ad | %s'
```

El líder de software extrae metadatos analíticos que permiten procesar dos indicadores estructurales determinantes:

Volatilidad del Código (Code Churn): Corresponde a la frecuencia y volumen con el que las líneas de código son insertadas, modificadas y eliminadas en archivos individuales durante un intervalo temporal definido. Un módulo que exhibe un churn desproporcionadamente elevado con intervenciones simultáneas de múltiples ingenieros en una misma iteración es un síntoma inequívoco de patologías de diseño. Esta conducta revela una violación frontal del Principio de Responsabilidad Única (SRP), indicando que el archivo concentra responsabilidades dispares y actúa como cuello de botella de integración.

Localización de Puntos Calientes (Hotspots): Al cruzar la dimensión de volatilidad histórica derivada de Git con la métrica de complejidad ciclomática de McCabe, se delimitan con exactitud los puntos calientes del sistema. Un componente simple con alto churn no reviste alarma estructural; sin embargo, un archivo que combina una lógica condicional densa y enrevesada con una tasa perpetua de modificaciones se consolida como el principal foco de fallos críticos en producción. La inspección forense de Git confiere al líder la visibilidad necesaria para aislar y refactorizar estos *hotspots* de manera previa al inicio de nuevos requerimientos, impidiendo que desestabilicen el cronograma planificado.

4. Madurez Organizacional y Estabilidad del Equipo

La viabilidad operativa de cualquier cronograma de ingeniería de software depende estrechamente de factores sociotécnicos vinculados a la estructura del equipo de trabajo y a su disciplina metodológica. Para controlar la incertidumbre en fases tempranas, el liderazgo técnico debe evaluar dos dimensiones fundamentales:

Mitigación del Factor Autobús (Bus Factor): Métrica de resiliencia organizacional que cuantifica el número mínimo de integrantes de un proyecto cuya pérdida o indisponibilidad repentina paralizaría por completo el avance del desarrollo o el mantenimiento de componentes vitales. Al procesar el mapa de autorías sobre el registro de confirmaciones en Git, el líder analiza la dispersión del conocimiento. Si el 80% del código de un subsistema crítico ha sido elaborado exclusivamente por una persona, el proyecto enfrenta un riesgo operacional inaceptable. El liderazgo debe intervenir promoviendo salvaguardas explícitas: revisiones de código obligatorias en pares (*pull request reviews*), programación en parejas (*pair programming*) en áreas complejas y rotación planificada de desarrolladores en diferentes módulos para descentralizar el conocimiento técnico.

Evaluación de Madurez del Proceso (Factor PMAT en COCOMO II): En el modelo algorítmico de costos COCOMO II (Boehm et al., 2000), el parámetro de madurez del proceso (PMAT) opera como un factor de escala exponencial que incide decisivamente en el esfuerzo total necesario para culminar una entrega. En el contexto práctico de desarrollo, el grado de madurez metodológica se manifiesta de manera directa en la rigurosidad y consistencia del historial de confirmaciones. La adhesión formal al estándar *Conventional Commits* (estableciendo prefijos semánticos estructurados como *feat*, *fix*, *refactor*, *docs* o *chore*) vinculada a los identificadores del sistema de gestión de tickets refleja un proceso maduro, con alta trazabilidad y predictibilidad. Por el contrario, un repositorio plagado de mensajes vagos ("cambios varios", "arreglo bug") denota un proceso informal e indisciplinado, lo cual obliga al líder técnico a ampliar los factores de escala de esfuerzo y los rangos de incertidumbre para compensar la baja madurez de ingeniería.

Conclusiones

En síntesis, la estimación técnica eficaz se fundamenta en sustituir fechas deterministas por rangos probabilísticos basados en el tiempo de ciclo y la Simulación de Monte Carlo. Asimismo, presupuestar de forma obligatoria un 15% a 20% para la deuda técnica previene el colapso arquitectónico y salvaguarda la predictibilidad operativa.

Por último, el análisis del historial de Git y el control del *bus factor* mitigan cuellos de botella de diseño y descentralizan el conocimiento en el equipo. En conclusión, liderar nuevos proyectos exige gestionar la incertidumbre a través de evidencia empírica cuantitativa y disciplina de procesos.